BigGoogle (BigQuery & Bigtable)

Gestión y Visualización de Datos Masivos

Profesor D. Ildefonso Iglesias Escribano
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA-BigGoogle (BigQuery vs Bigtable) - Actividad.pdf» (Campus Virtual UPO).

Objetivos

• Conocer y utilizar el almacén de datos Google BigQuery.

• Ser capaz de aplicar técnicas de análisis de datos en Google BigQuery.

• Aplicar modelos ligeros sobre datos en Google BigQuery para hacer predicciones.

• Saber cómo utilizar la documentación ofrecida por GCP.

• Defender y argumentar la decisión de cuándo es aconsejable utilizar Google BigQuery o Cloud Bigtable.

Conceptos Teóricos Previos

1. Google Cloud Platform

De forma previa, este trabajo parte de que el alumno ya dispone de una cuenta de usuario en Google Cloud Platform que fue creada para los ejercicios de la clase “Google Cloud, servicio y pipeline mediante ejemplos”. En caso de no disponer de ella, el alumno deberá crear dicha cuenta siguiendo las instrucciones que se dieron en aquella clase.

2. Google BigQuery

Recordemos que cuando hablamos de Google BigQuery nos estamos refiriendo a un almacén de datos en la nube, que es rápido, económico, de gran escalabilidad y totalmente administrado, el cual nos permite realizar análisis con aprendizaje automático integrado.

Toda la documentación relativa a esta herramienta que Google Cloud Platform tiene a disposición de los usuarios se encuentra en https://docs.cloud.google.com/bigquery/docs.

3. Cloud Bigtable

Por otro lado, tengamos en cuenta que cuando nos referimos a Cloud Bigtable, estamos hablando de un servicio de base de datos NoSQL a escala de petabytes, totalmente administrado y que permite la ejecución de grandes cargas de trabajo analítico y operativo.

Toda la documentación relativa a esta herramienta que Google Cloud Platform tiene a disposición de los usuarios se encuentra en https://docs.cloud.google.com/bigtable/docs.

Ejercicio/Problemas

Los siguientes ejercicios pueden tener sus fundamentos en cuestiones tratadas durante la clase o relativas a cuestiones existentes en la propia documentación de Google Cloud Platform para cada una de sus herramientas. El objetivo de todas ellas es que el alumno aprenda a analizar y decidir las mejores alternativas ante las necesidades planteadas.

E1. (90 minutos – 1,5 puntos) Explore los conjuntos de datos públicos de los que dispone Google BigQuery, y seleccione el conjunto denominado "GHCN Monthly" que se encuentra dentro de la temática "clima". A continuación, lea detenidamente la "Visión general" del conjunto de datos en su página de detalle e impórtelo sobre la herramienta. Responda a las siguientes preguntas, adjuntando una captura de pantalla del conjunto de datos importado correctamente sobre BigQuery:

1. ¿Qué conclusión ha sacado del conjunto de datos tras leer detenidamente la descripción del mismo?

2. ¿Qué nuevos conjuntos de datos se han vinculado a su panel de navegación?

3. Visualice las tablas y datos del conjunto de datos que ha seleccionado y que se encuentran en "bigquery-public-data.ghcn_m.*". Preste especial atención a las descripciones de las columnas de cada tabla en la sección "schema". Tras observar los datos y las descripciones que existen en cada campo de las tablas:

a. Describa detalladamente qué podemos encontrar en cada una de las tablas que componen este conjunto de datos.

b. Tras esto, proponga alguna utilidad de análisis y predicción que pueda tener el tratamiento de estos datos.

E2. (50 minutos – 0,5 puntos) Lea la documentación oficial de Google Cloud Platform (https://docs.cloud.google.com/bigquery/docs/locations) que habla sobre las consideraciones a tener en cuenta en el momento de elegir la mejor localización para un conjunto de datos. Es importante que preste especial atención a la sección “Location considerations”. A continuación, resuma las consideraciones que más le hayan llamado la atención en una redacción limitada a 100 palabras como máximo.

Tras la lectura anterior, cree un conjunto de datos (dataset) nuevo en su Google BigQuery completamente vacío y decida un nombre para él. Tenga en cuenta que en ejercicios posteriores, este conjunto de datos nos servirá para exportar a él nuevas tablas con datos, a partir de consultas que se ejecutarán sobre el conjunto de datos estudiado en el ejercicio anterior, así que preste especial atención a la ubicación en la que lo crea. Adjunte una captura de pantalla del conjunto de datos creado correctamente sobre BigQuery en la que se aprecie la información del dataset: id, fecha de creación, fecha de modificación, localización, etc.

E3. (120 minutos – 3 puntos) Esta actividad tiene dos opciones en función del perfil que se ajuste más a sus intereses. Tenga en cuenta que es libre de hacer las dos opciones, aunque sólo se exige una, pero en ese caso el tiempo estimado será el doble.

Opción 1 (Perfil analista):

A continuación, podrá observar dos sentencias SQL de selección de los datos del conjunto de datos público que importó en el ejercicio 1:

1. SELECT id, year, IF (value1 = -9999, NULL, value1) as value1, IF (value2 = -9999, NULL, value2) as value2, IF (value3 = -9999, NULL, value3) as value3, IF (value4 = -9999, NULL, value4) as value4, IF (value5 = -9999, NULL, value5) as value5, IF (value6 = -9999, NULL, value6) as value6, IF (value7 = -9999, NULL, value7) as value7, IF (value8 = -9999, NULL, value8) as value8, IF (value9 = -9999, NULL, value9) as value9, IF (value10 = -9999, NULL, value10) as value10, IF (value11 = -9999, NULL, value11) as value11, IF (value12 = -9999, NULL, value12) as value12 FROM `bigquery-public-data.ghcn_m.ghcnm_tmin`

2. SELECT id, year, IF (value1 = -9999, NULL, value1) as value1, IF (value2 = -9999, NULL, value2) as value2, IF (value3 = -9999, NULL, value3) as value3, IF (value4 = -9999, NULL, value4) as value4, IF (value5 = -9999, NULL, value5) as value5, IF (value6 = -9999, NULL, value6) as value6, IF (value7 = -9999, NULL, value7) as value7, IF (value8 = -9999, NULL, value8) as value8, IF (value9 = -9999, NULL, value9) as value9, IF (value10 = -9999, NULL, value10) as value10, IF (value11 = -9999, NULL, value11) as value11, IF (value12 = -9999, NULL, value12) as value12 FROM `bigquery-public-data.ghcn_m.ghcnm_tmax` WHERE value1 != -9999 OR value2 != -9999 OR value3 != -9999 OR value4 != -9999 OR value5 != -

9999 OR value6 != -9999 OR value7 != -9999 OR value8 != -9999 OR value9 != -9999 OR value10 != -9999 OR value11 != -9999 OR value12 != -9999

En primer lugar, lea detenidamente las funciones y operaciones soportadas por Google BigQuery en el siguiente enlace https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/functions-all. Una vez leído:

1. Ejecute cada una de estas sentencias en el editor de Google BigQuery, adjuntando una captura de pantalla donde se aprecie la correcta ejecución de cada consulta.

2. Analice las sentencias anteriores y observe los resultados de cada una. A continuación, exprese con detalle cuál es la finalidad de cada sentencia y qué se pretende con cada selección de datos.

3. Exprese otra posible selección de datos que se podría realizar sobre el conjunto de datos analizado y la finalidad que perseguiría con esa selección. No es necesario que escriba la sentencia SQL de selección, tan solo que la defina de forma clara y concisa. No serán válidas variantes de las consultas expresadas en el enunciado de este ejercicio, por lo que se espera que se describa una selección de datos completamente diferente a las del enunciado.

4. Por cada sentencia anterior del enunciado, exporte los resultados a una nueva tabla en Google BigQuery, mediante la opción “Save Results”. Esa nueva tabla deberá estar en el dataset que ha creado en el ejercicio 2. Adjunte capturas de pantalla de la correcta creación de cada tabla donde se aprecie la pestaña “details” (id, fecha de creación, fecha de modificación, expiración, localización, etc.), así como otra captura con los datos almacenados en cada una de ellas de la pestaña “preview”.

Opción 2 (Perfil técnico):

Lea detenidamente las funciones y operaciones soportadas por Google BigQuery en el siguiente enlace https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/functions-all. A continuación, a partir de las tablas originales del conjunto de datos público que importó en el ejercicio 1, y junto a las funciones y operaciones disponibles en Google BigQuery, realice una sentencia de selección (no se admiten sentencias iguales o variantes a las reflejadas para la Opción 1 de este ejercicio), de tal forma que le permita obtener los datos de una de las tablas (o combinación de más de una tabla) mejor estructurados de cara a poder analizarlos. Para la sentencia que ha generado y ejecutado:

1. Adjunte una captura de pantalla de la sentencia correctamente ejecutada en BigQuery.

2. Cuál es la finalidad de la consulta de datos construida y qué ha pretendido con esta selección de datos.

3. Exporte los resultados obtenidos con la ejecución de esa sentencia a una nueva tabla en Google BigQuery, mediante la opción “Save Results”. Esa nueva tabla deberá estar en el dataset que ha creado en el ejercicio 2. Adjunte capturas de pantalla de la correcta creación de la tabla donde se aprecie la pestaña “details” (id, fecha de creación, fecha de modificación, expiración, localización, etc.), así como otra captura con los datos almacenados en ella de la pestaña “preview”.

E4. (90 minutos – 2 puntos) Esta actividad tiene dos opciones en función del perfil que se ajuste más a sus intereses. Tenga en cuenta que es libre de hacer las dos opciones, aunque sólo se exige una, pero en ese caso el tiempo estimado será el doble.

Opción 1 (Perfil analista):

Visualice el siguiente vídeo (https://www.youtube.com/watch?v=ZVgt1-LfWW4) que habla sobre buenas prácticas a tener en cuenta cuando usamos Google BigQuery como almacén de datos. A continuación:

1. Resuma los contenidos e ideas abordados en el vídeo en una redacción limitada a 400 palabras como máximo.

2. Describa una propuesta de análisis predictivo sobre los datos de temperaturas máximas tratados en el ejercicio anterior.

Opción 2 (Perfil técnico):

De forma previa, solo en el caso de que en el ejercicio 3 escogiese la opción de "Perfil técnico", deberá ejecutar la segunda sentencia que se visualiza en el ejercicio anterior para el "Perfil analista" y, tras obtener los resultados, procederá a exportarlos a una nueva tabla en Google BigQuery mediante la opción "Save Results". Esa nueva tabla deberá estar en el dataset que ha creado en el ejercicio 2. Adjunte capturas de pantalla de la correcta ejecución de la consulta y de la correcta creación de la tabla, así como de los datos almacenados en ella.

Céntrese ahora solo en la nueva tabla que ha generado a partir de la exportación de los datos de la sentencia a la que hacemos referencia en el párrafo anterior. Esta tabla se corresponde con la medición de temperaturas máximas, por lo que, teniendo este dato en cuenta, proceda a:

• Generar un modelo haciendo uso de Google BigQuery ML para la predicción del tiempo en un determinado año. Adjunte captura de pantalla de la sentencia de entrenamiento del modelo, así como de la correcta creación del modelo en Google BigQuery, donde se aprecien las métricas finales del entrenamiento.

• Construir la sentencia de predicción sobre el modelo generado y ejecutarlo sobre algún dato de la tabla para comprobar sus resultados. Adjunte captura de pantalla de la sentencia de predicción del modelo ejecutada en Google BigQuery.

E5. (90 minutos – 1,5 puntos) Piense en el caso de uso completo que se ha generado a través de las actividades anteriores, pero centrándonos únicamente en las actividades de carga y análisis de los datos. Responda a las siguientes cuestiones:

• ¿Hubiese sido más correcto, en vez de exportar los datos a otro conjunto de datos en Google BigQuery, haberlo hecho a una instancia con tablas de Cloud Bigtable? ¿Por qué?

• En caso negativo, con qué propósito futuro cree que hubiese sido necesario almacenar dichos datos en Cloud Bigtable.

• Piense en la tabla de temperaturas máximas que ha generado y escriba, haciendo uso del comando "cbt", aquellos comandos necesarios para: ○ Generar la tabla dentro de una instancia con el mismo nombre que el conjunto de datos en Google BigQuery. Adjunte captura de pantalla de la correcta ejecución de todos los comandos en Bigtable. ○ Añadir las columnas de la tabla de Google BigQuery a la nueva tabla generada en Cloud Bigtable. Adjunte captura de pantalla de la correcta ejecución de todos los comandos en Bigtable.

IMPORTANTE: Tras concluir el ejercicio número 5, asegúrese de que destruye de manera completa la instancia de Bigtable que haya creado para evitar cargos superiores a los disponibles mediante los créditos libres de GCP.

E6. (100 minutos – 1,5 puntos) Lea detenidamente el siguiente enunciado:

Una red de sensores situados en una gran plantación de 100 hectáreas para abastecimiento de riego recoge continuamente millones de registros de los diferentes sensores para medir la temperatura, humedad, conductividad de la tierra, etc. A partir de estos datos, es necesario elaborar una solución para detectar anomalías en tiempo real, a la vez que se realizan análisis sobre ciertos conjuntos de datos para predecir averías y mantenimientos.

A continuación, piense, diseñe y justifique una solución a partir de Google BigQuery y/o Cloud Bigtable para dar solución a la necesidad que se está planteando, teniendo en cuenta que existen otras herramientas en Google Cloud Platform que visteis en la sesión “Google Cloud, servicio y pipeline mediante ejemplos”, que pueden ayudar en la misma para el tratamiento y procesamiento de datos. Tenga en cuenta que se espera el diseño de una solución (dibujo de la arquitectura de componentes y comunicación entre los mismos), así como el texto de descripción y justificación de la solución.

Fecha y Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.
- Este documento deberá contener todas las respuestas a cada uno de los ejercicios, siendo imprescindible para que los ejercicios puedan ser calificados de forma correcta, adjuntar todas las capturas de pantalla solicitadas para asegurar que los ejercicios se han ejecutado sobre Google Cloud Platform, así como aquellas que el alumno crea necesarias, ayudando a comprender lo expuesto en cada ejercicio y a confirmar la realización completa de los mismos.
- La entrega se ha de llevar a cabo durante el día 13 de abril de 2026.

Bibliografía

1. Google Cloud: https://docs.cloud.google.com/docs.
2. Google BigQuery Documentation: https://docs.cloud.google.com/bigquery/docs.
3. Cloud Bigtable Documentation: https://docs.cloud.google.com/bigtable/docs.

RESOLUCIÓN

Ejecución real. Todo lo que sigue se ha ejecutado en un proyecto de Google Cloud propio (`mbda-gvdm-2026`, con el crédito de la prueba gratuita): BigQuery (consultas, tablas guardadas y modelo de BigQuery ML) y Cloud Bigtable (instancia, tabla y datos con `cbt` desde Cloud Shell). La instancia de Bigtable se **eliminó al terminar** para no consumir crédito.

E1. Conjunto de datos público «GHCN Monthly» (`bigquery-public-data.ghcn_m`)

Cómo importarlo: en la consola de BigQuery, *Add data → Public datasets* (Marketplace), buscar «GHCN Monthly» (temática *Climate*), abrir la ficha y pulsar *View dataset*. El proyecto `bigquery-public-data` queda **marcado con estrella** en el panel *Explorer* y el conjunto `ghcn_m` se puede consultar sin copiarlo: se paga sólo por los bytes leídos.

1) Conclusión tras leer la descripción. GHCN-M (*Global Historical Climatology Network – Monthly*) es la base de datos de NOAA/NCEI con **temperaturas medias mensuales de miles de estaciones terrestres de todo el mundo**, algunas con series desde el siglo XVIII. Los datos proceden de fuentes nacionales e internacionales, pasan controles de calidad y se usan como referencia en estudios de cambio climático, como el seguimiento de la temperatura global de NOAA. Es un conjunto **histórico, de gran cobertura temporal y espacial y con valores ausentes frecuentes**. Hay que limpiarlo antes de analizarlo.

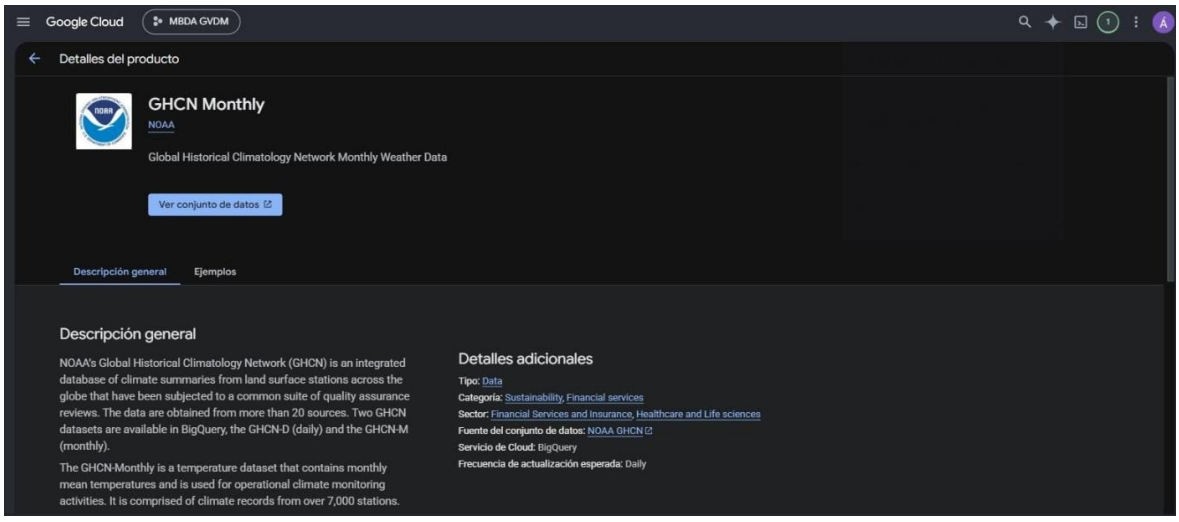


Figura 1. Ficha de «GHCN Monthly» en el Marketplace de Google Cloud (NOAA): más de 7.000 estaciones y más de 20 fuentes.

2) Conjuntos vinculados al panel. Al abrir el conjunto desde el Marketplace se añade al panel *Explorer* el proyecto **`bigquery-public-data`**, con todos sus conjuntos públicos (entre ellos `ghcn_m` y el diario `ghcn_d`), además de los conjuntos propios del proyecto propio.

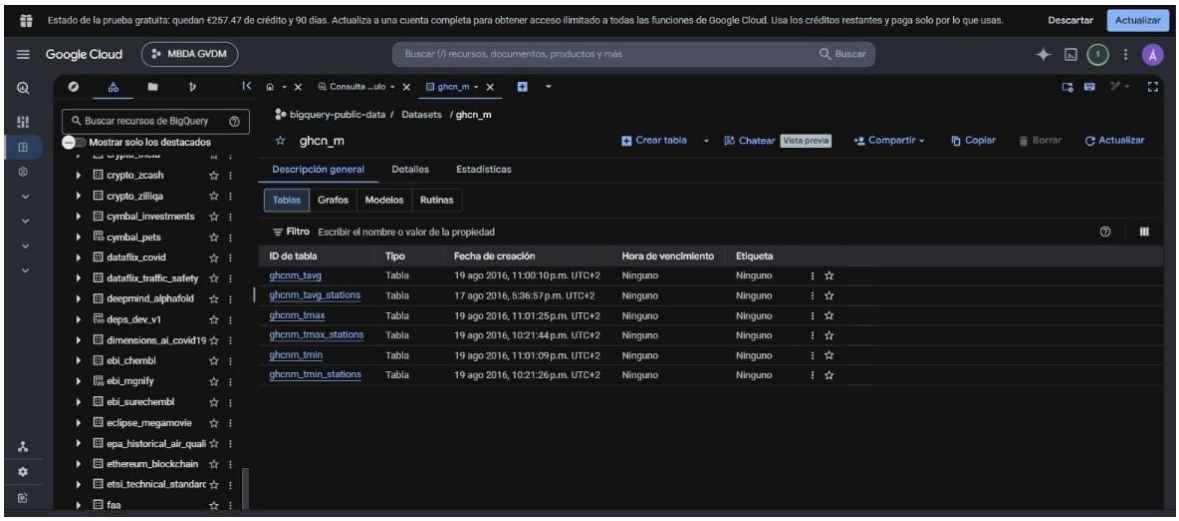


Figura 2. Conjunto `bigquery-public-data.ghcn_m` en BigQuery Studio: seis tablas, tres de medidas y tres de estaciones.

3) Tablas de `ghcn_m` (comprobadas en la consola; creadas en agosto de 2016, ubicación US):

Tabla	Contenido
`ghcnm_tavg`	Temperatura **media** mensual. Una fila por estación y año.
`ghcnm_tmax`	Media mensual de las temperaturas **máximas** diarias: 333.294 filas (≈ 52 MB), 5.809 estaciones, años 1840-2019.
`ghcnm_tmin`	Media mensual de las temperaturas **mínimas** diarias.
`ghcnm_tavg_stations`, `ghcnm_tmax_stations`, `ghcnm_tmin_stations`	Inventario de estaciones (5.809 en `tmax`): `id`, `latitude`, `longitude`, `stnelev` (altitud), `name` y metadatos del entorno: `popcls`/`popsiz` (población), `topo`, `stveg`, `ocndis` (distancia al mar), `airstn` (aeropuerto), `towndis`, `grveg`…

Columnas de las tablas de medidas: `id` (estación), `year` (año), `element` (TAVG/TMAX/TMIN) y, para cada mes 1..12, `valueN` (valor, en centésimas de grado en GHCN-M v3), `dmflagN` (indicador de medida), `qcflagN` (control de calidad: NULL si el valor lo supera) y `dsflagN` (fuente del dato). Los tipos son `id` INTEGER (11 dígitos), `year` INTEGER, `valueN` INTEGER y *flags* STRING. Los valores están en **centésimas de grado** (2860 = 28,6 °C). El valor **-9999 indica dato ausente**, y por eso las consultas del E3 lo sustituyen por NULL.

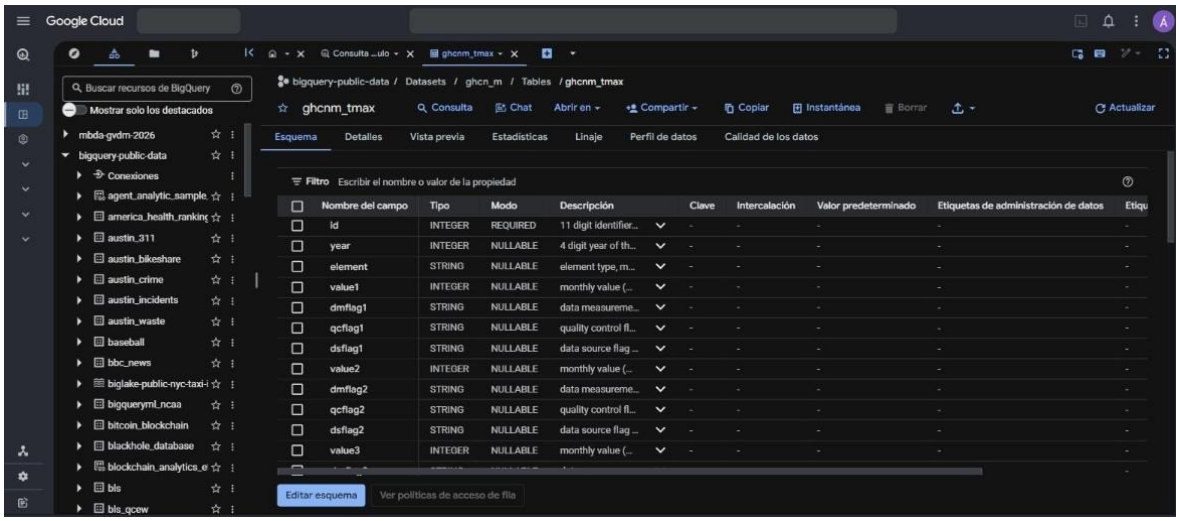


Figura 3. Esquema de `ghcnm_tmax`.

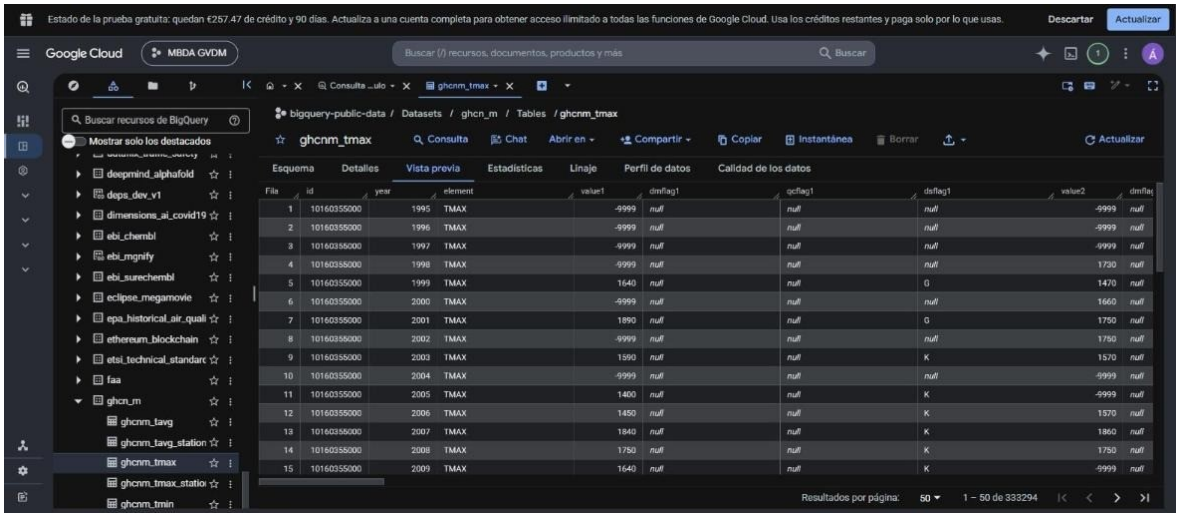


Figura 4. Vista previa de `ghcnm_tmax`: valores en centésimas de grado y -9999 en los meses sin dato.

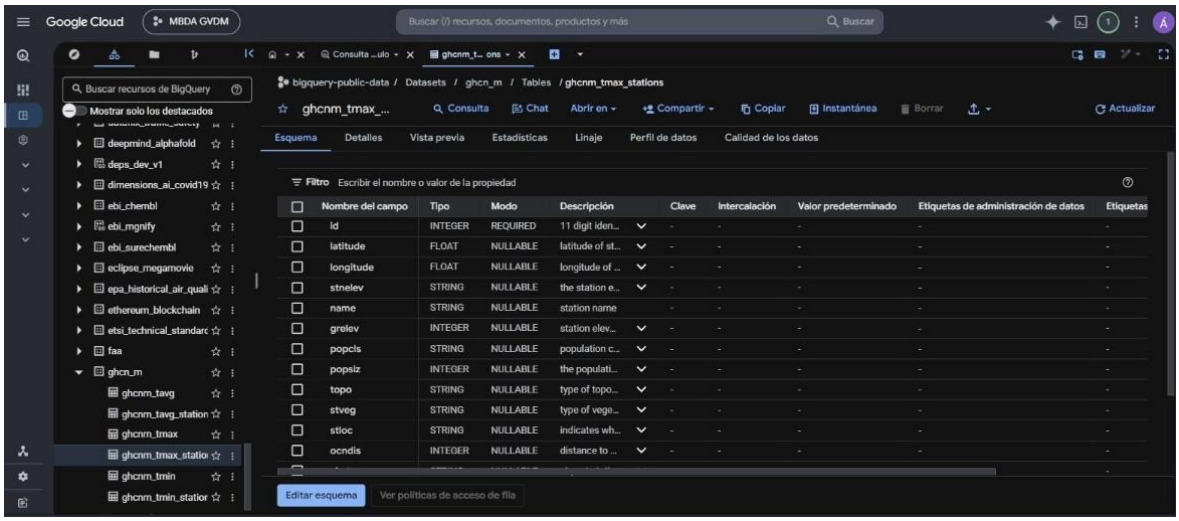


Figura 5. Esquema de `ghcnm_tmax_stations`.

b) Utilidades de análisis y predicción: tendencias de calentamiento por región o década; anomalías respecto a la media 1961-1990; frecuencia de meses extremos; comparación entre estaciones urbanas y rurales (efecto isla de calor, con los metadatos de estaciones);

predicción de la temperatura mensual de años futuros por estación (regresión o series temporales); y su uso como variable en modelos agrícolas o de demanda energética (calefacción y refrigeración).

E2. Localización del conjunto de datos

Consideraciones más relevantes (≤ 100 palabras)

La ubicación (región o multirregión) se fija al crear el conjunto de datos y no puede cambiarse después: habría que copiarlo. Una consulta sólo puede leer tablas de la misma ubicación y escribir resultados en ella, así que conviene situar el conjunto junto a los datos con los que se combinará. Lo mismo vale para los buckets de Cloud Storage desde los que se carga o exporta. Las multirregiones (US, EU) ofrecen más disponibilidad; las regiones permiten cumplir requisitos de residencia de datos y reducir latencia. El precio también varía según la ubicación.

(93 palabras)

Creación del conjunto de datos

Los conjuntos de `bigquery-public-data` están en la **multirregión US**. Como en el E3 se guardarán en este conjunto resultados de consultas sobre `ghcn_m`, **el nuevo conjunto debe crearse en `US` (multirregión)**. Si se creara en `EU` o en `europe-southwest1`, la opción *Save results* a una tabla de ese conjunto fallaría por estar en otra ubicación. Nombre elegido: **`mbda_ghcn`**.

```
-- Consola: Explorer › (proyecto) › Create dataset › Dataset ID = mbda_ghcn,
Location type = Multi-region, US
-- Equivalente en SQL:
CREATE SCHEMA `mbda-gvdm-2026.mbda_ghcn` OPTIONS (location = "US", description =
"GVDM - actividad BigGoogle");
```

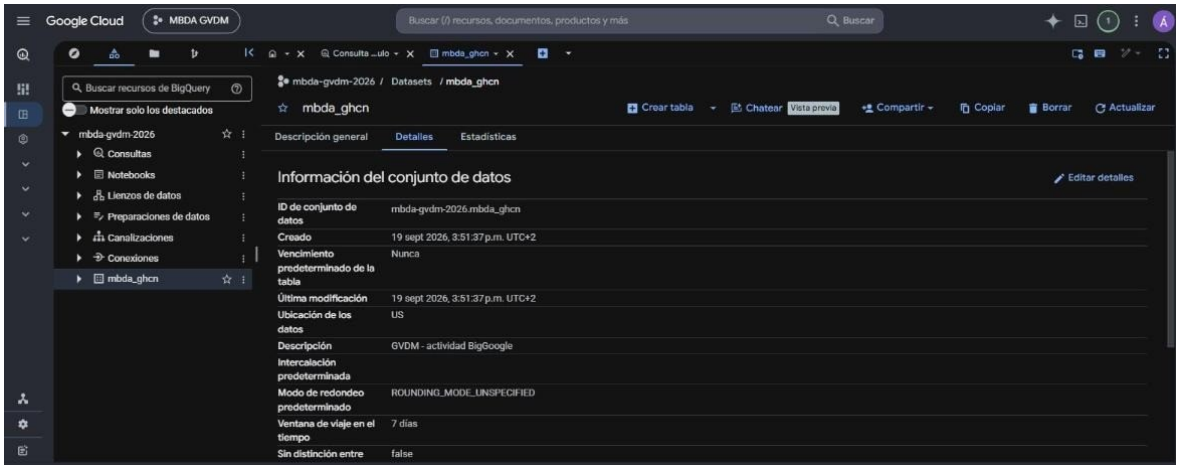


Figura 6. Pestaña Detalles del conjunto `mbda_ghcn`: creado el 19/09/2026, ubicación US, sin vencimiento de tablas.

E3. Opción 1 (perfil analista)

1) Ejecución de las dos sentencias en el editor de BigQuery:

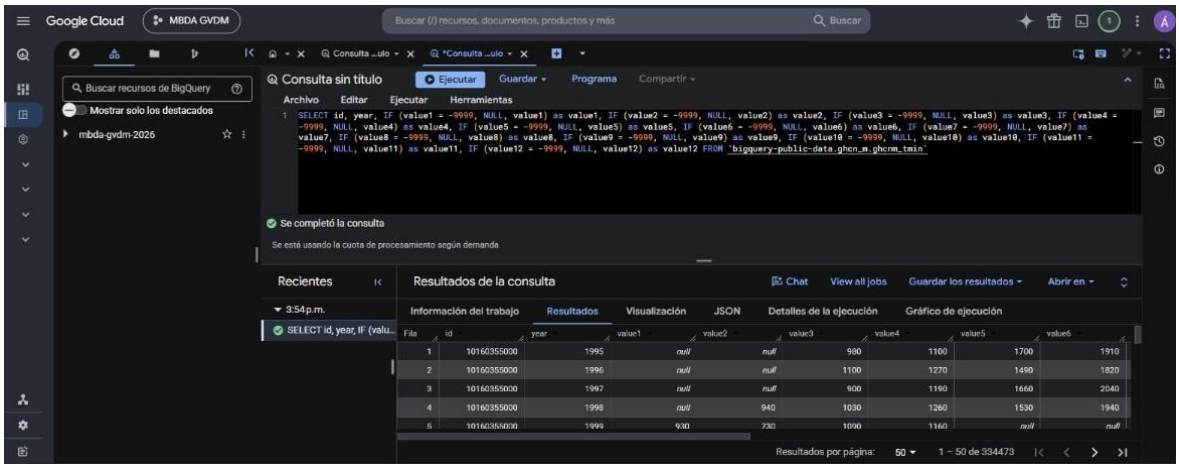


Figura 7. Sentencia 1 sobre `ghcnm_tmin`: 334.473 filas, con NULL donde había -9999.

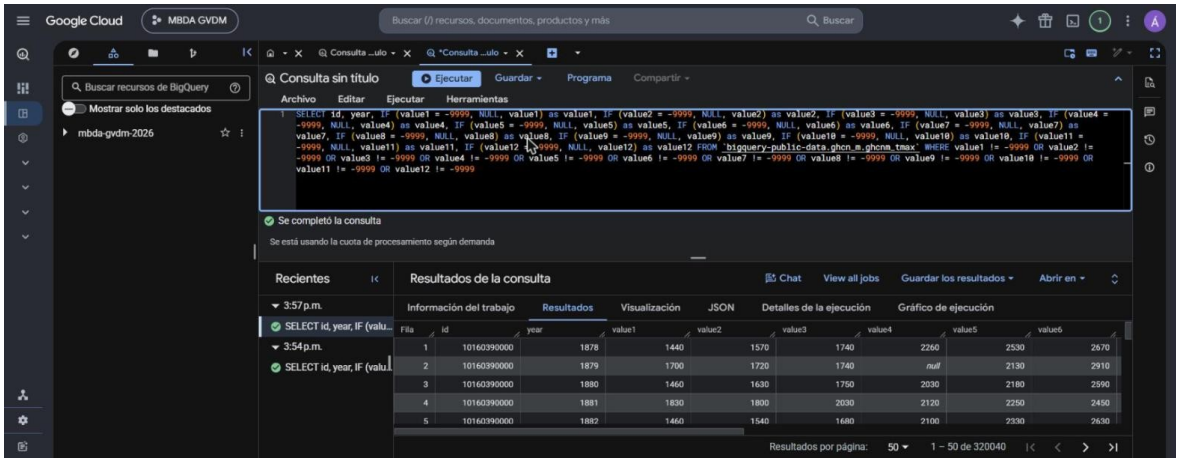


Figura 8. Sentencia 2 sobre `ghcnm_tmax` con el filtro WHERE: 320.040 filas.

2) Finalidad de cada sentencia:

- **Sentencia 1 (tabla `ghcnm_tmin`):** selecciona **todas** las filas (estación, año y los 12 valores mensuales) de las temperaturas mínimas y **sustituye el código -9999 por NULL** con `IF(valueN = -9999, NULL, valueN)`. Es un paso de **limpieza**: con NULL, las funciones de agregación (`AVG`, `MIN`…) ignoran los meses sin dato en lugar de tratarlos como -99,99 °C, que falsearía cualquier media. Además descarta las columnas de *flags* y `element`, dejando una tabla ancha más manejable. Conserva las filas en las que faltan los 12 meses.
- **Sentencia 2 (tabla `ghcnm_tmax`):** hace la misma limpieza sobre las temperaturas máximas y añade un `WHERE` con una cadena de `OR`: sólo conserva los registros de estación-año que tengan **al menos un mes con dato válido**. Elimina las filas completamente vacías, que no aportan información y ocupan espacio: de las 333.294 filas de `ghcnm_tmax` quedan **320.040** (se descartan 13.254 estación-año sin ningún mes válido). La tabla resultante es más pequeña y está lista para el análisis y para entrenar el modelo del E4.

3) Otra selección de datos posible: unir `ghcnm_tavg` con su tabla de estaciones para quedarse con las **estaciones de la Península Ibérica** (latitud 36-44 °N, longitud -9,5 a 3,5 °E) con al menos 30 años de datos, calcular por estación y **década** la temperatura media anual

(sólo años con los 12 meses válidos y sin *flag* de calidad) y obtener la **tendencia de calentamiento** (°C por década). Finalidad: cuantificar el calentamiento regional y comparar estaciones costeras con interiores o urbanas con rurales. Es una selección distinta porque cruza con metadatos, filtra geográficamente y agrega en el tiempo.

4) Exportación con *Guardar los resultados → Tabla de BigQuery* al conjunto `mbda_ghcn`, en las tablas `tmin_limpia` y `tmax_limpia`:

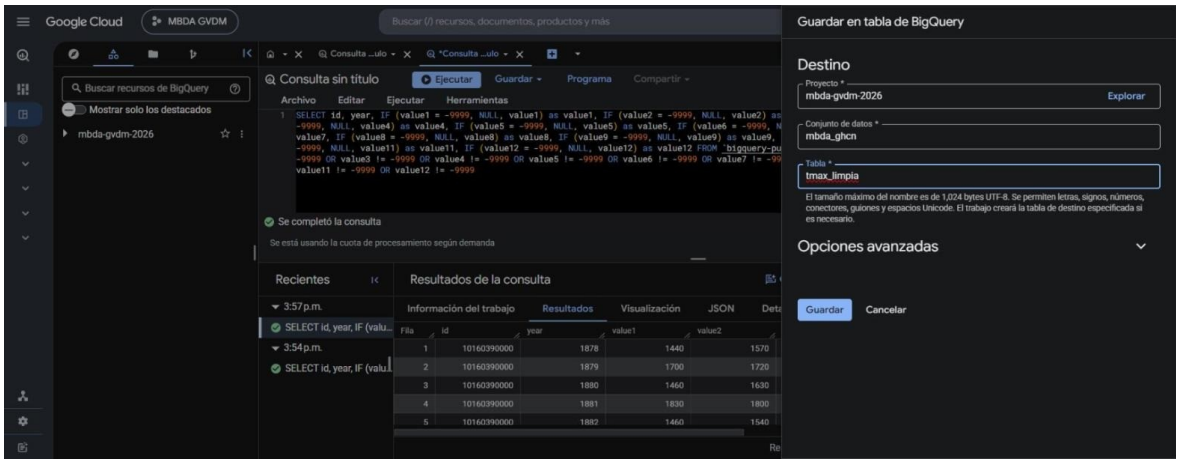


Figura 9. Guardar los resultados de la sentencia 2 como tabla `mbda_ghcn.tmax_limpia`.

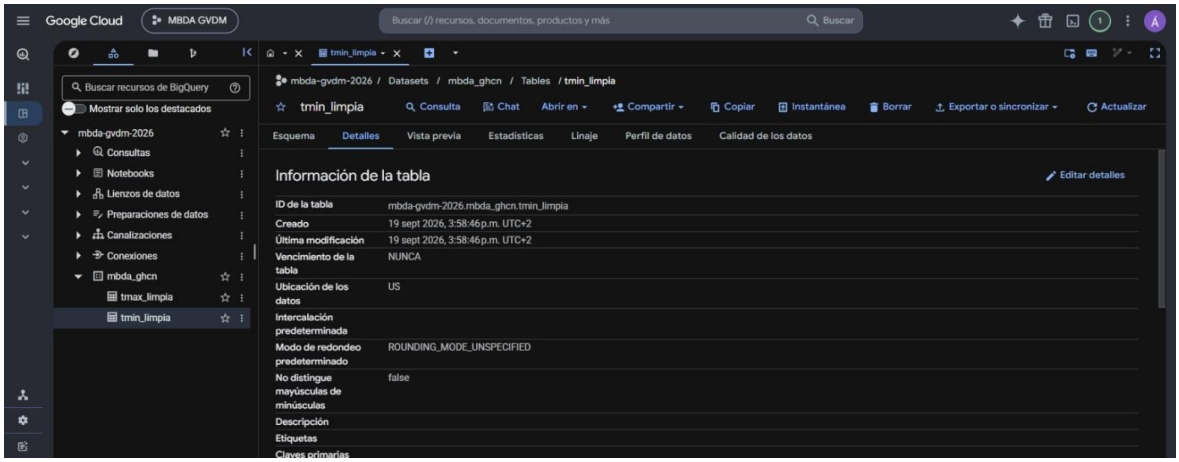


Figura 10. Detalles de `tmin_limpia`: creada el 19/09/2026 a las 15:58:46, sin vencimiento, ubicación US.

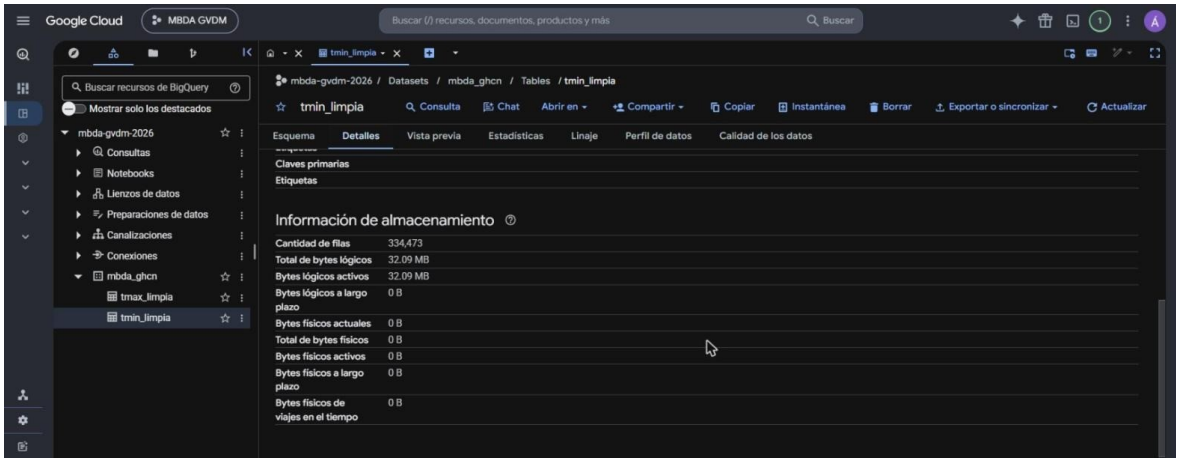


Figura 11. Información de almacenamiento de `tmin_limpia`: 334.473 filas y 32,09 MB lógicos.

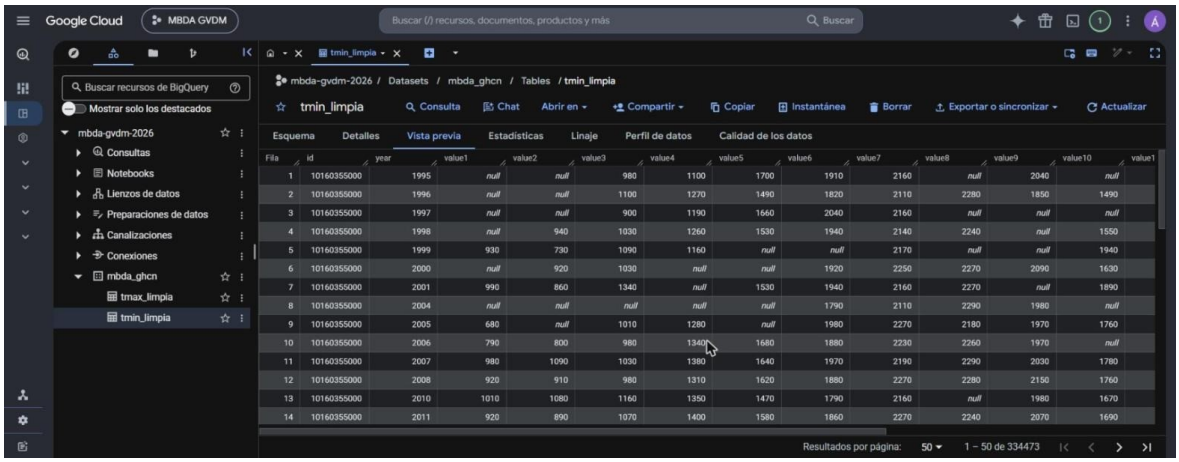


Figura 12. Vista previa de `tmin_limpia`.

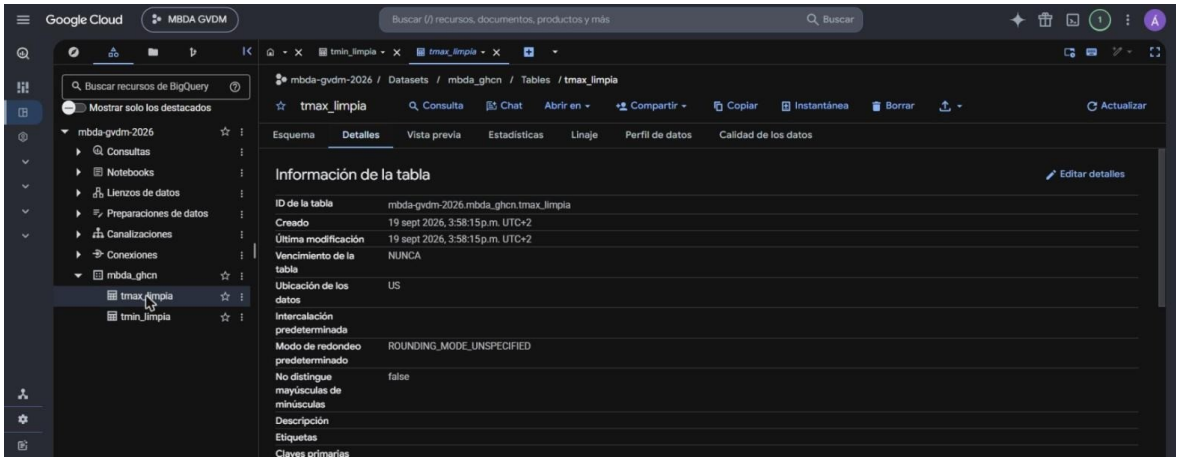


Figura 13. Detalles de `tmax_limpia`: creada el 19/09/2026 a las 15:58:15, ubicación US (320.040 filas, 32,25 MB).

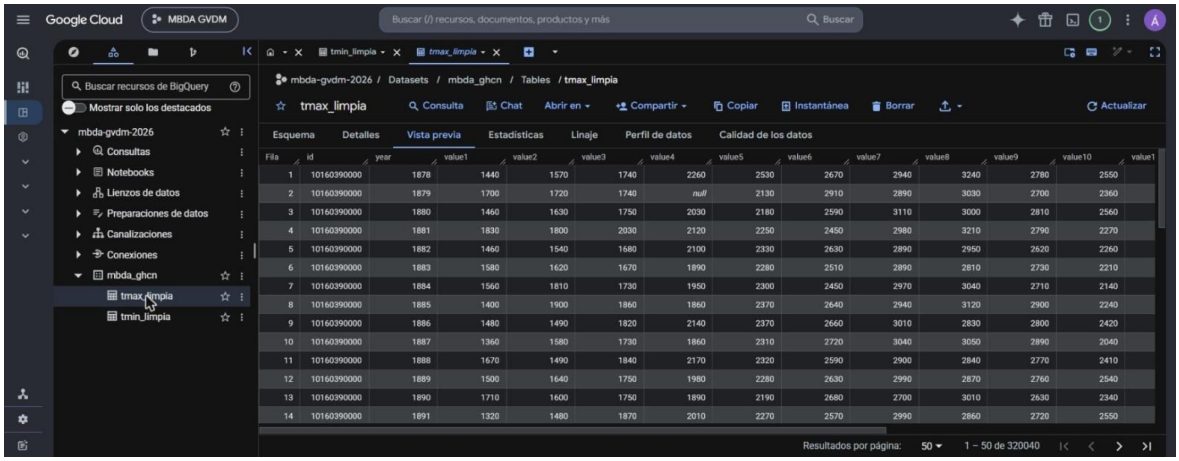


Figura 14. Vista previa de `tmax_limpia`.

E3. Opción 2 (perfil técnico): sentencia propuesta

La tabla original es «ancha» (12 columnas de valor por fila), incómoda para analizar. La consulta la **pivota a formato largo**: una fila por estación, año y mes, con fecha real, valor en °C y posición de la estación. Así se puede agrupar, filtrar por fecha o representar series temporales directamente:

```
CREATE OR REPLACE TABLE `mbda_ghcn.tmax_largo` AS
```

```
SELECT
  t.id,
  st.latitude, st.longitude, st.name,
  DATE(t.year, m.mes, 1)            AS fecha,
  t.year                            AS anio,
  m.mes,
  m.valor / 100.0                   AS tmax_c        -- centésimas de grado → °C
FROM `bigquery-public-data.ghcn_m.ghcnm_tmax` AS t,
UNNEST([
  STRUCT(1 AS mes, t.value1 AS valor, t.qcflag1 AS qc), (2, t.value2, t.qcflag2), (3,
t.value3, t.qcflag3),
  (4, t.value4, t.qcflag4), (5, t.value5, t.qcflag5), (6, t.value6, t.qcflag6), (7,
t.value7, t.qcflag7),
  (8, t.value8, t.qcflag8), (9, t.value9, t.qcflag9), (10, t.value10, t.qcflag10),
  (11, t.value11, t.qcflag11), (12, t.value12, t.qcflag12)
]) AS m
LEFT JOIN `bigquery-public-data.ghcn_m.ghcnm_tmax_stations` AS st USING (id)
WHERE m.valor != -9999 AND (m.qc IS NULL OR m.qc = '');
```

Finalidad: dejar una tabla **normalizada y georreferenciada**, sin ausentes ni valores que no pasan el control de calidad, apta para series temporales, mapas y modelos. Se ha creado directamente en `mbda_ghcn` con `CREATE TABLE … AS`:

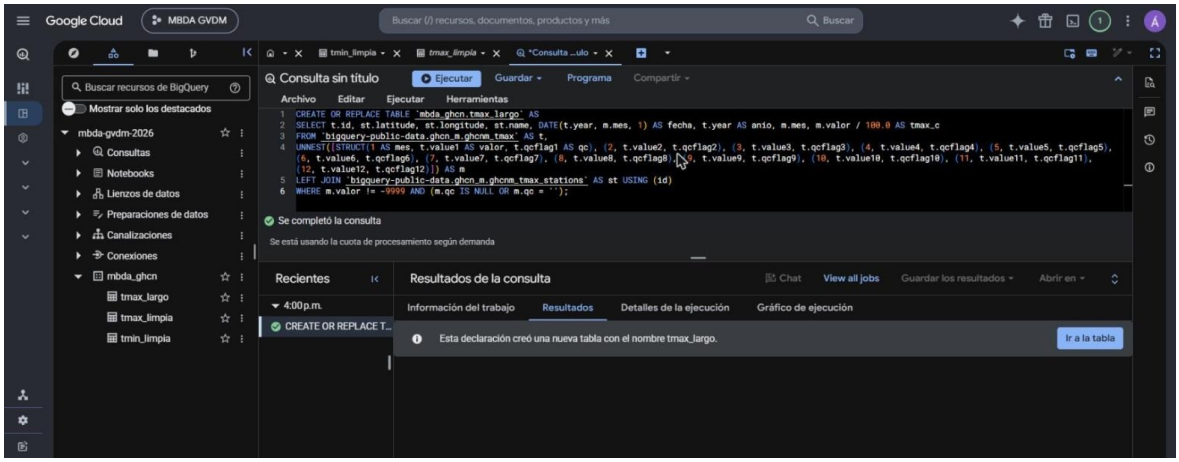


Figura 15. Creación de `mbda_ghcn.tmax_largo` (pivotado a formato largo con UNNEST).

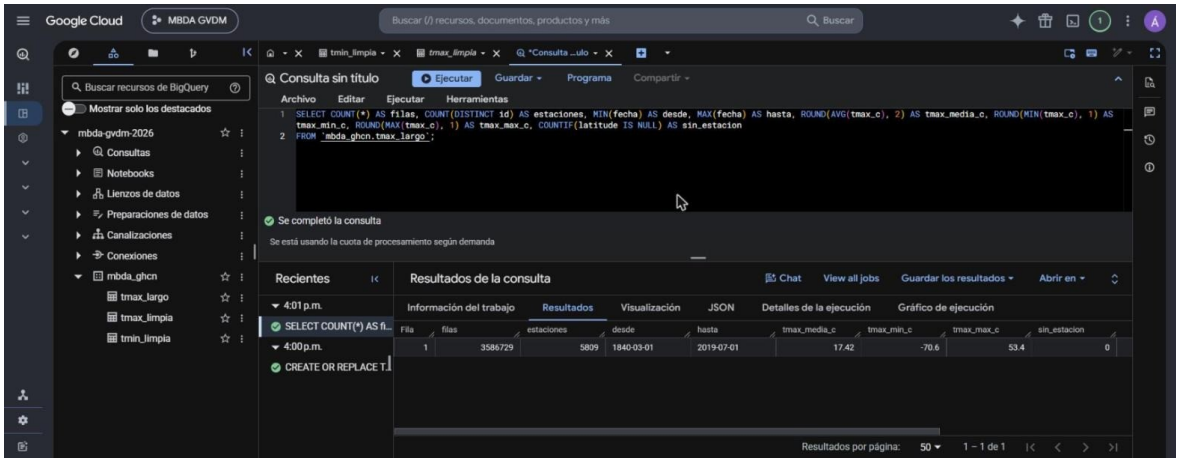


Figura 16. Resumen de la tabla resultante.

Resultado: **3.586.729 filas** (estación-mes válidas) de **5.809 estaciones**, de marzo de 1840 a julio de 2019. La Tmax media es 17,42 °C y todas las filas tienen estación asociada. El mínimo (-70,6

°C) y el máximo (53,4 °C) son extremos que conviene revisar: el primero es físicamente dudoso para una media mensual de máximas. Es una muestra de que, además de los *flags* de calidad, se pueden añadir reglas de rango.

E4. Opción 1 (perfil analista)

1) Resumen del vídeo «Data Warehousing With BigQuery: Best Practices» (Cloud Next '19) (≤ 400 palabras)

Ryan McDowell, ingeniero de datos de Google Cloud, repasa los problemas de los almacenes de datos tradicionales: no escalan con el crecimiento de los datos, exigen mucho personal de administración, licencias caras y paradas para actualizar. BigQuery los resuelve con una arquitectura sin servidor, totalmente gestionada, en la que almacenamiento y cómputo están desacoplados y escalan por separado. Un *shuffle* en memoria remota entre etapas acelera las consultas y tolera fallos de trabajadores. El almacenamiento es gestionado: comprimido y cifrado por defecto, repartido en miles de discos y replicado entre zonas o regiones.

Las buenas prácticas que propone son: cargar preferentemente en **Avro**, que es el formato más rápido porque se lee en paralelo (los CSV y JSON comprimidos con gzip son los más lentos); preferir **ELT a ETL**, transformando con SQL dentro de BigQuery; organizar las tablas en capas **raw → staging → reporting**, usando `MERGE` para la captura de cambios y las dimensiones lentas de tipo 2. En el diseño del esquema recomienda migrar primero el modelo en estrella tal cual y optimizar después. Aconseja **desnormalizar** con campos anidados y repetidos (pedidos con sus líneas, sesiones con sus eventos) y **particionar** por fecha las tablas de hechos, exigiendo filtro de partición para evitar lecturas completas. También recomienda aplicar **clustering** sobre las columnas más filtradas, respetando su orden, y sustituir las claves subrogadas secuenciales por UUID o *hashes* SHA-256.

Sobre la gestión de cargas, explica que el planificador reparte los *slots* de forma equitativa entre consultas y que las **reservas** permiten priorizar proyectos, como los cuadros de mando de dirección, sin crear silos, porque los *slots* ociosos se comparten. También aconseja analizar los registros de auditoría en BigQuery para controlar costes.

En la segunda parte, Alban Perillat-Merceroz cuenta el caso de Teads, empresa de publicidad que procesa 10.000 millones de eventos al día con un equipo de 10 personas. Sustituyó una arquitectura lambda (Kafka, S3, Spark, Storm y Cassandra) por Dataflow y BigQuery con enfoque ELT. Sus agregaciones horarias comprimen los datos 70 veces y agrupan 100 millones de filas en 1 minuto y 40 segundos. Eligió la tarifa plana de *slots*, con costes predecibles, y aísla cargas mediante proyectos y reservas jerárquicas. Concluye que BigQuery le dio rendimiento, simplicidad y datos accesibles con SQL, y le permitió centrarse en el producto en lugar de en la infraestructura.

(389 palabras)

2) Propuesta de análisis predictivo sobre las temperaturas máximas

Objetivo: predecir la temperatura máxima media mensual de una estación para los próximos años y detectar estaciones con calentamiento anómalo. **Datos:** la tabla `tmax_limpia`, pivotada a formato largo (E3, opción 2) y unida a los metadatos de estación. **Variables:** año (tendencia), mes (estacionalidad, codificado como categoría o con seno y coseno), latitud,

longitud, altitud, distancia al mar, entorno urbano o rural, y la media de la estación en los 5 años anteriores (retardos). **Modelo:** como primera aproximación, regresión lineal o *boosted trees* en BigQuery ML; para series de una estación, un modelo ARIMA_PLUS por estación (`time_series_id_col = id`), que capta tendencia y estacionalidad. **Validación temporal:** entrenar hasta 2010 y evaluar 2011-2020 (MAE en °C). **Uso:** mapas de calentamiento previsto y alertas de estaciones con residuos persistentes, que pueden indicar un cambio de instrumento o de entorno.

E4. Opción 2 (perfil técnico): BigQuery ML

```
-- Entrenamiento: regresión lineal de la Tmax mensual en función del año, el mes (por
hemisferio) y la ubicación
CREATE OR REPLACE MODEL `mbda_ghcn.modelo_tmax`
OPTIONS (model_type = 'linear_reg', input_label_cols = ['tmax_c'],
         data_split_method = 'SEQ', data_split_col = 'fecha', data_split_eval_fraction
= 0.2) AS
SELECT tmax_c, anio,
       CONCAT(IF(latitude >= 0, 'N', 'S'), '-', CAST(mes AS STRING)) AS
hemisferio_mes,
       latitude, ABS(latitude) AS abs_lat, longitude, fecha
FROM `mbda_ghcn.tmax_largo`
WHERE anio >= 1950;
```

Decisiones: el mes se combina con el hemisferio (`N-7`, `S-7`…) porque julio es verano en el norte e invierno en el sur, y un único coeficiente por mes no lo capta. La distancia al ecuador (`abs_lat`) recoge el gradiente térmico con la latitud. `fecha` sólo se usa para la división secuencial (`SEQ`): el 20 % más reciente de los datos se reserva para evaluar.

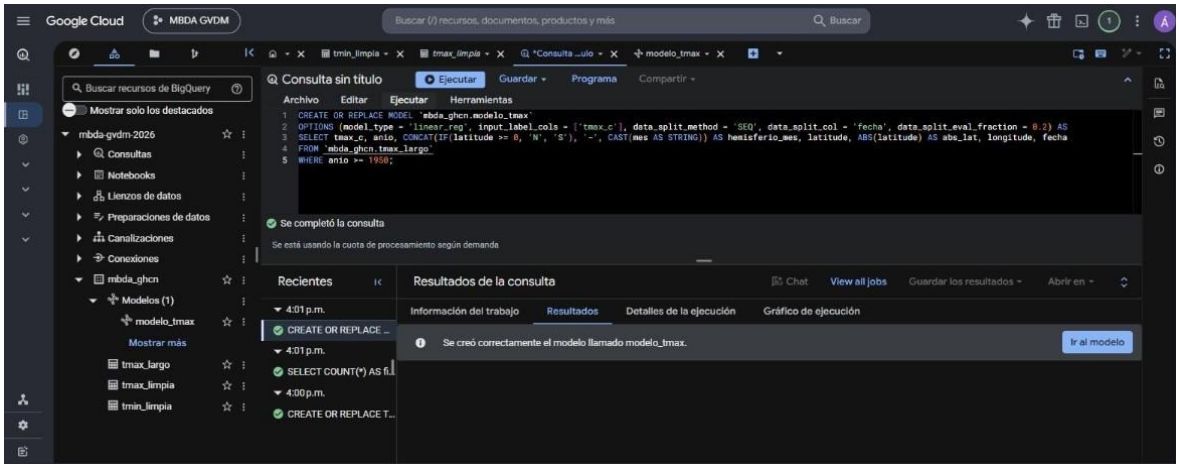


Figura 17. Sentencia de entrenamiento ejecutada: modelo `modelo_tmax` creado.

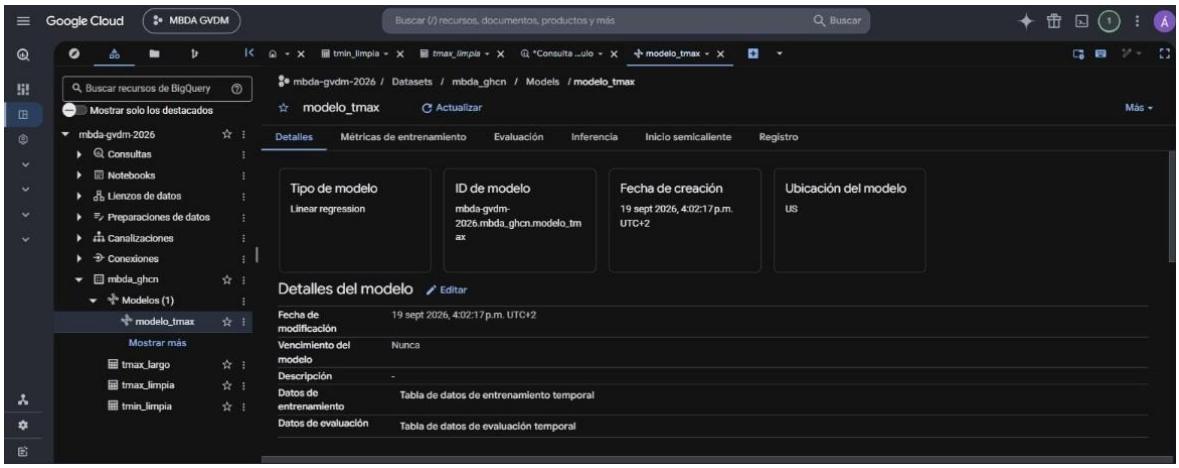


Figura 18. Detalles del modelo (regresión lineal, ubicación US).

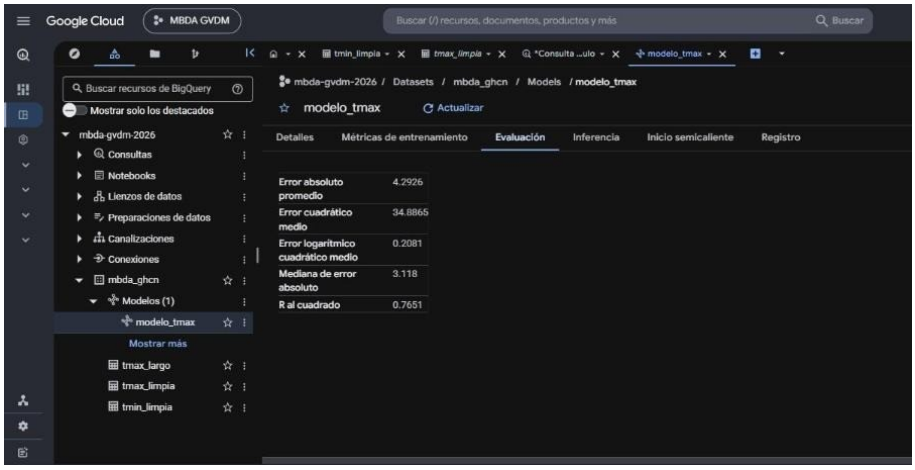


Figura 19. Pestaña Evaluación con las métricas sobre el conjunto de evaluación.

Métrica (evaluación)	**Valor**	**Lectura**
Error absoluto medio (MAE)	4,29 °C	Error típico de la predicción
Error cuadrático medio	34,89	RMSE ≈ 5,9 °C: hay errores grandes
Mediana del error absoluto	3,12 °C	La mitad de las predicciones fallan menos de 3,1 °C
R^2	0,765	Explica el 76,5 % de la varianza

Predicción para 2030 en una estación concreta (Madrid/Barajas) y comparación con la media observada 1990-2019:

```
WITH pred AS (SELECT id, name, mes, ROUND(predicted_tmax_c, 1) AS tmax_prevista_2030
  FROM ML.PREDICT(MODEL `mbda_ghcn.modelo_tmax`,
    (SELECT id, name, mes, 2030 AS anio,
            CONCAT(IF(latitude >= 0, 'N', 'S'), '-', CAST(mes AS STRING)) AS
hemisferio_mes,
            latitude, ABS(latitude) AS abs_lat, longitude
     FROM (SELECT DISTINCT id, name, latitude, longitude FROM `mbda_ghcn.tmax_largo`
           WHERE REGEXP_CONTAINS(name, r'SEVILL|MADRID')),
          UNNEST(GENERATE_ARRAY(1, 12)) AS mes)))
SELECT pred.*, obs.tmax_media_1990_2019
FROM pred LEFT JOIN (SELECT id, mes, ROUND(AVG(tmax_c), 1) AS tmax_media_1990_2019
                     FROM `mbda_ghcn.tmax_largo` WHERE anio BETWEEN 1990 AND 2019
                     GROUP BY id, mes) AS obs USING (id, mes)
ORDER BY id, mes;
```

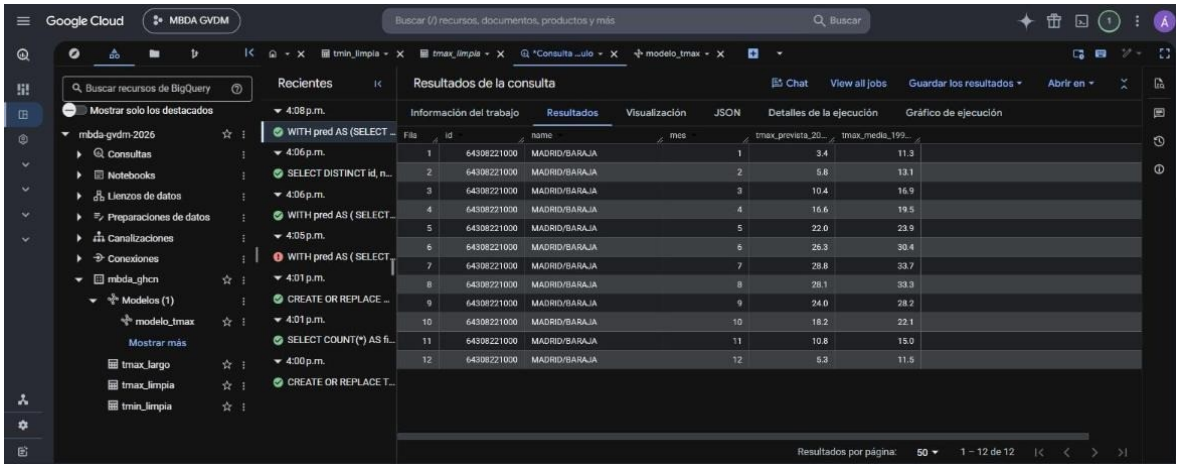


Figura 20. ML.PREDICT para Madrid/Barajas (la única estación de la búsqueda con datos de Tmax en GHCN-M): previsión 2030 frente a media observada 1990-2019.

Interpretación honesta. El modelo reproduce bien el ciclo anual (mínimo en enero, máximo en julio-agosto), pero **subestima Madrid entre 2 y 8 °C** (por ejemplo, 3,4 °C previstos en enero frente a 11,3 °C observados). Una regresión lineal global sólo con latitud y longitud no conoce la altitud ni la continentalidad ni el clima local, así que predice el «promedio» de su franja de latitud. Sirve como modelo base. Para usarlo de verdad habría que (1) añadir `stnelev`, `ocndis` y el entorno de la estación; (2) incluir la media histórica de la propia estación como variable (retardos); o (3) entrenar un **ARIMA_PLUS por estación** (`time_series_id_col = id`), que modela tendencia y estacionalidad de cada serie y es la opción natural para prever años futuros de una estación concreta.

E5. BigQuery frente a Cloud Bigtable

¿Habría sido más correcto exportar a Bigtable en vez de a otro conjunto de BigQuery? No. Lo que se hace en el caso de uso es **analítico**: consultas SQL que recorren y agregan muchas filas y columnas, limpieza, *joins* con estaciones y entrenamiento de modelos con BigQuery ML. BigQuery es un almacén columnar sin servidor pensado exactamente para eso. Se paga por datos leídos y la carga es por lotes y con volumen modesto (decenas o cientos de MB). Bigtable es una base de datos NoSQL clave-valor de columna ancha para **lecturas y escrituras de baja latencia (ms) y muy alto rendimiento por clave de fila**. No tiene SQL completo ni agregaciones nativas eficientes, y cobra por nodos aprovisionados 24/7. Además, habría obligado a diseñar una clave de fila y a mover los datos otra vez a BigQuery para analizarlos.

¿Con qué propósito futuro convendría almacenarlos en Bigtable? Si los datos pasaran a **servir una aplicación operacional**: por ejemplo, una API o *app* que devuelva al instante la serie de una estación (clave `id#año`) a miles de usuarios concurrentes. O si se incorporaran **lecturas en tiempo real de sensores** a alta frecuencia (millones de escrituras por segundo), guardando la serie temporal reciente en Bigtable y enviando periódicamente los agregados a BigQuery para el análisis. Es la arquitectura del E6.

Comandos cbt (ejecutados en Cloud Shell)

Los identificadores de instancia de Bigtable sólo admiten minúsculas, dígitos y guiones (6-33 caracteres), así que el conjunto `mbda_ghcn` se traduce como **`mbda-ghcn`**. Instancia mínima de desarrollo: 1 nodo HDD en `us-central1-b`.

```
# Configuración de la CLI (cbt viene instalado en Cloud Shell)
echo "project = mbda-gvdm-2026" > ~/.cbtrc
echo "instance = mbda-ghcn" >> ~/.cbtrc

# Crear la instancia (id, nombre visible, id de clúster, zona, nodos, tipo de disco)
cbt createinstance mbda-ghcn "mbda ghcn" mbda-ghcn-c1 us-central1-b 1 HDD
cbt listinstances && cbt listclusters
```

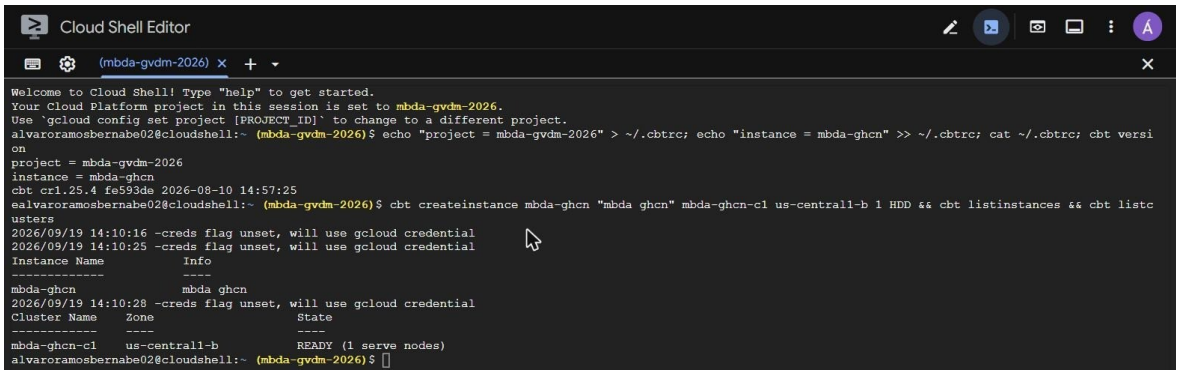


Figura 21. Creación de la instancia: clúster mbda-ghcn-c1 en estado READY con 1 nodo.

```
# Tabla y familia de columnas (una sola versión por celda)
cbt createtable tmax_limpia
cbt createfamily tmax_limpia medidas
cbt setgcpolicy tmax_limpia medidas maxversions=1
cbt ls tmax_limpia

# Exportar filas de la tabla de BigQuery (20 años de Madrid/Barajas) a CSV
bq query --use_legacy_sql=false --format=csv \
  'SELECT * FROM mbda_ghcn.tmax_limpia WHERE id = 64308221000 ORDER BY year DESC LIMIT 20'
| tail -n +2 > filas.csv

# Cargar cada fila en Bigtable. Clave = id#year; las columnas (qualifiers) se crean al
escribir;
# los meses sin dato (NULL) simplemente no se escriben
while IFS=, read -ra f; do
  args=(medidas:id=${f[0]} medidas:year=${f[1]})
  for i in $(seq 1 12); do v=${f[$((i+1))]}; [ -n "$v" ] && args+=(medidas:value$i=$v);
done
  cbt set tmax_limpia "${f[0]}#${f[1]}" "${args[@]}"
done < filas.csv
cbt count tmax_limpia
cbt read tmax_limpia start=64308221000#2017 end=64308221000#2019
columns=medidas:year,medidas:value1,medidas:value7
```



Figura 22. Familia `medidas` con política de 1 versión y exportación desde BigQuery de las 20 filas a cargar.

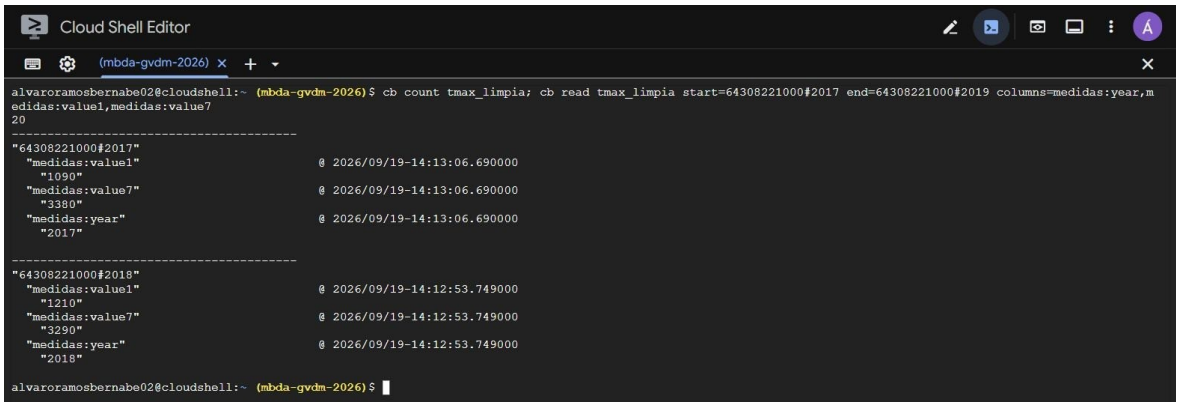


Figura 23. Tras la carga: 20 filas en Bigtable y lectura por rango de clave (2017-2018), filtrando columnas.

En Bigtable **no se declaran columnas**: sólo familias. Cada `cbt set` crea las columnas `medidas:value1…value12` de esa fila, y una fila con meses sin dato simplemente no tiene esas celdas (almacenamiento disperso). Para cargar la tabla completa (320.040 filas) no se usaría un bucle, sino una exportación de BigQuery a Cloud Storage y la plantilla de Dataflow «Cloud Storage Avro/Parquet to Bigtable», o la exportación directa de BigQuery a Bigtable (`EXPORT DATA … OPTIONS(format="CLOUD_BIGTABLE")`).

```
# Limpieza: borrar tabla e instancia para no consumir crédito
cbt deletetable tmax_limpia
cbt deleteinstance mbda-ghcn
gcloud bigtable instances list
```

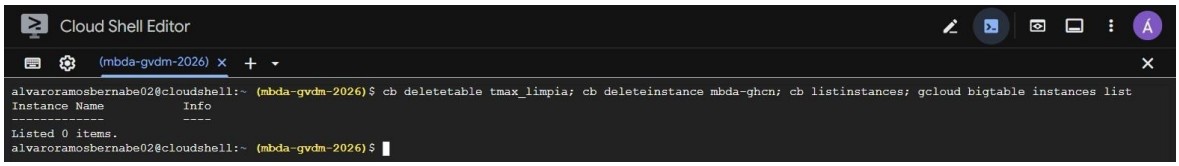


Figura 24. Instancia eliminada: `gcloud bigtable instances list` devuelve 0 elementos.

E6. Solución para la red de sensores de la plantación

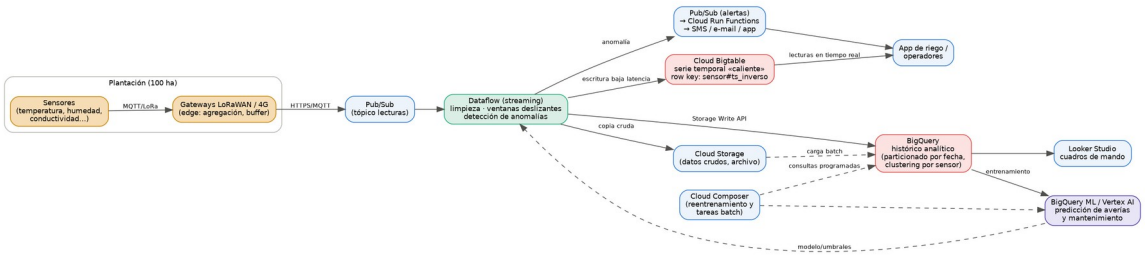


Figura 25. Arquitectura propuesta sobre Google Cloud Platform.

Requisitos: (a) ingerir continuamente millones de lecturas (temperatura, humedad, conductividad…); (b) **detectar anomalías en tiempo real**; (c) **análisis histórico y predicción** de averías y mantenimiento. Son dos patrones distintos, operacional y analítico, así que la solución combina **Bigtable y BigQuery**, cada uno para lo que mejor hace:

- **Ingesta:** los sensores envían por LoRaWAN o 4G a *gateways* que agregan y almacenan temporalmente si se pierde la conexión. Los *gateways* publican en **Pub/Sub**, un bus de mensajes gestionado, escalable, que desacopla productores y consumidores y retiene mensajes ante picos. Cloud IoT Core ya no existe, así que la conexión se hace con MQTT/HTTPS directamente o mediante un *broker* de partners.
- **Procesamiento en* streaming*: Dataflow (Apache Beam) valida y limpia las lecturas y calcula estadísticas por ventanas deslizantes, por ejemplo 5 min por sensor. Aplica la detección de anomalías: umbrales físicos, puntuación z frente a la media móvil y un modelo entrenado en Vertex AI o BigQuery ML. Si detecta una anomalía, publica una alerta en otro tópico de Pub/Sub, que una Cloud Run Function convierte en notificación (SMS, correo o* app**) o en una orden al sistema de riego.
- **Cloud Bigtable – datos calientes:** guarda la serie temporal reciente con clave `sensor#timestamp_invertido`, así las lecturas más recientes de un sensor quedan contiguas. Admite millones de escrituras por segundo con latencia de milisegundos y sirve a la *app* de operadores y a los paneles en tiempo real («estado actual de cada sensor»).
- **BigQuery – histórico analítico:** Dataflow escribe también en BigQuery con la *Storage Write API*. Las tablas se **particionan por fecha** y se aplica **clustering por sensor y variable**, que son las buenas prácticas del vídeo del E4. Ahí se hacen el análisis de tendencias, las correlaciones entre variables y averías y los informes.
- **Predicción de averías:** con el histórico y el registro de incidencias de mantenimiento se entrenan en **BigQuery ML** (o Vertex AI) modelos de clasificación (fallo en los próximos N días) y de series temporales (ARIMA_PLUS para la deriva de sensores). **Cloud Composer** programa el reentrenamiento y las consultas batch, y el modelo actualizado alimenta la detección de Dataflow.
- **Cloud Storage** archiva los datos crudos a bajo coste (reproceso y auditoría), y **Looker Studio** ofrece cuadros de mando sobre BigQuery.

Justificación: los servicios son gestionados y escalan de forma automática, así que no hay servidores que administrar. Se separa el camino de baja latencia (Pub/Sub → Dataflow → Bigtable y alertas) del analítico (BigQuery y ML), que es la arquitectura Kappa/Lambda simplificada que recomienda Google. El coste es proporcional: Bigtable se dimensiona por nodos según el caudal de escritura, BigQuery cobra por almacenamiento y consultas, y los datos antiguos pasan a almacenamiento a largo plazo o a Cloud Storage.

Referencias

[1] BigQuery – Dataset locations. https://docs.cloud.google.com/bigquery/docs/locations

[2] NOAA NCEI – Global Historical Climatology Network Monthly (GHCN-M). https://www.ncei.noaa.gov/products/land-based-station/global-historical-climatology-network-monthly

[3] R. McDowell y A. Perillat-Merceroz, «Data Warehousing With BigQuery: Best Practices», Google Cloud Next ’19. https://www.youtube.com/watch?v=ZVgt1-LfWW4

[4] BigQuery ML – CREATE MODEL (linear_reg, ARIMA_PLUS). https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create

[5] Cloud Bigtable – cbt CLI reference. https://docs.cloud.google.com/bigtable/docs/cbt-reference